arXiv:2404.15159v3 [cs.CL] 20 Jul 2024

MIXLORA: Enhancing Large Language Models Fine-Tuning with LoRA-based Mixture of Experts

Dengchun Li*, Yingzi Ma*, Naizheng Wang*, Zhengmao Ye*, Zhiyuan Cheng[1], Yinghao Tang*, Yan Zhang[3], Lei Duan*, Jie Zuo*, Cal Yang[2], and Mingjie Tang*

*Sichuan University, Chengdu, China
[1]Purdue University, West Lafayette, USA
[2]Emory University, Atlanta, USA
[3]Nanyang Technological University, Singapore
mikecovlee@163.com, {g19myz, pherenice1125, yezhengmaolove}@gmail.com, cheng443@purdue.edu,, {yinghaotang2001, yanzhang.jlu}@gmail.com,, {leiduan, zuojie}@scu.edu.cn,, j.carlyang@emory.edu, tangrock@gmail.com

Abstract

Fine-tuning Large Language Models (LLMs) is a common practice to adapt pre-trained models for specific applications. While methods like LoRA have effectively addressed GPU memory constraints during fine-tuning, their performance often falls short, especially in multi-task scenarios. In contrast, Mixture-of-Expert (MoE) models, such as Mixtral 8x7B, demonstrate remarkable performance in multi-task learning scenarios while maintaining a reduced parameter count. However, the resource requirements of these MoEs remain challenging, particularly for consumer-grade GPUs with less than 24GB memory. To tackle these challenges, we propose MIXLORA, an approach to construct a resource-efficient sparse MoE model based on LoRA. MIXLORA inserts multiple LoRA-based experts within the feed-forward network block of a frozen pre-trained dense model and employs a commonly used top-k router. Unlike other LoRA-based MoE methods, MIXLORA enhances model performance by utilizing independent attention-layer LoRA adapters. Additionally, an auxiliary load balance loss is employed to address the imbalance problem of the router. Our evaluations show that MIXLORA improves about 9% accuracy compared to state-of-the-art PEFT methods in multi-task learning scenarios. We also propose a new high-throughput framework to alleviate the computation and memory bottlenecks during the training and inference of MOE models. This framework reduces GPU memory consumption by 40% and token computation latency by 30% during both training and inference.

1 Introduction

Instruction fine-tuning of Large Language Models (LLMs) [1; 2; 3; 4; 5] for various downstream tasks has achieved impressive proficiency in Natural Language Processing (NLP) [6; 7; 8]. As the scale of parameters increases, LLMs have been demonstrated to be able to identify complex linguistic patterns, thereby enabling the emergence of powerful cross-task generalization capabilities [9]. The paradigm of instruction tuning leads to a trade-off between the computational resources required and the performance achieved on downstream tasks, which has been a valuable facet.

To substantially reduce the computational and memory resources required by full parameter fine-tuning processes, Parameter-Efficient Fine-Tuning (PEFT) methods have emerged [10; 11; 12; 13; 14; 15]. Among these methods, Low-Rank Adaptation (LoRA) [15], a popular PEFT approach,

Preprint. Under review.

Release	Method	Model Details: Attention layer	Model Details: FFN layer	Training Details & Motivation: Router	Training Details & Motivation: Load balance	Training Details & Motivation: Solved Problems
2023.10	MOELoRA	Plugin ③	Plugin ④	Top-1 (*task*)	None	Medical Multi-Task Learning
2023.12	LoRAMoE	Vanilla ①	Plugin ④	Top-All (*token*)	None	World Knowledge Forgetting
2023.12	MoCLE	Plugin ③	Plugin ④	Top-1 (*token*)	None	Multi-Modal Task Conflicts
2024.02	MoELoRA	Plugin ③	Plugin ④	Top-K (*token*)	Contrastive Loss	Parameter-Efficient MoE
2024.02	MOLA	Plugin ③	Plugin ④	Top-K (*token*)	None	Parameter-Efficient MoE
2024.02	MoRAL	Vanilla ①	Plugin ④	Top-K (*token*)	None	Life-long Learning
2024.03	MoA	Plugin ③	Plugin ④	Top-K (*task*)	None	Multi-task Learning
2024.04	**MixLoRA**	Fusion ②	Fusion ⑤	Top-K (*token*)	Auxiliary Loss	Parameter-Efficient MoE

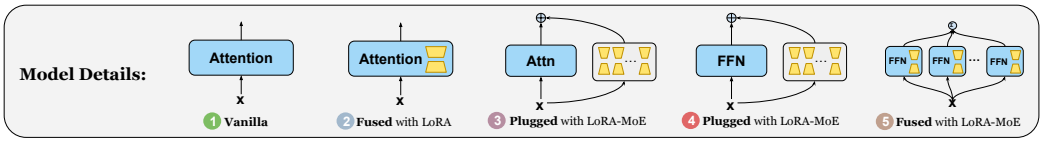


Figure 1: The timeline of public LoRA-MoE methods' release dates, including the detailed model information on the position of integration, how to train with the LoRA-MoE method (router and load balance), and the problems they aim to solve.

offers performance comparable to full fine-tuning across various downstream tasks while demanding less computational effort. This is achieved by introducing low-rank adaption matrices to simulate the gradient updates while keeping pre-trained model weights frozen. However, its performance still falls short of full fine-tuning [16; 17]. Recent studies, such as LoRA+ [18] and DoRA [19], aim to enhance the performance of LoRA by optimizing the parameter updating process. Despite these improvements, LoRA-based methods face challenges in handling multiple tasks simultaneously due to limited trainable parameters and the problem of catastrophic forgetting [20; 21], which hampers cross-task generalization in LLMs. To mitigate this, some approaches attempt to integrate additional components into existing methods. For instance, AdapterFusion [22] and LoRAHub [21] introduce task-specific LoRA modules or experts, combining knowledge into domain adapters through additional attention fusion and element-wise composition. However, these additional components often add complexity to the training process, limiting their applicability compared to simpler PEFT methods like LoRA and its variants.

A promising solution is to design an architecture that combines LoRA's resource-saving features with the versatility of MoE models. This approach involves adding multiple LoRA modules as experts in transformer sublayers (i.e., attention and feedforward layers) [23; 24; 20], using a router to assign experts to tokens [23; 25; 26; 27; 28; 20], and incorporating an expert balancing loss to prevent uneven token distribution [23; 25]. As shown in Fig. 1, most methods in this direction focus on solving specific-domain problems such as medical multi-task learning [28] and world knowledge forgetting [26]. Only MoELoRA [24] and MOLA [29] are proposed to enhance the general capability of multi-task learning for LLMs. However, they only focus on single-task learning while ignoring multi-task learning, leading to certain limitations. We observe that these LoRA-MoE models plug multiple LoRA modules into a single self-attention or FFN block to form the MoE structure (③ and ④ in Figure 1), while the high-performance pre-trained MoE models, such as Mixtral 8x7B [30], often use multiple FFN based expert networks during forward propagation. Moreover, research on vanilla transformers [31] indicates that employing MoE only on the FFN block can be more efficient than on both self-attention and FFN blocks, with fine-tuning the attention layer with LoRA can further enhance MoE models [32]. Additionally, these LoRA-MoE methods introduce multiple LoRAs within a single transformer block, providing opportunities to improve the computational efficiency using Multi-LoRA parallel computing techniques [33].

Inspired by these, we propose MIXLORA, which fuses multiple LoRAs with the shared FFN layer and employs them to store updated parameters for each expert during fine-tuning, thereby aligning it more with conventional MoE models such as Mixtral 8x7B [30]. Additionally, we employ LoRA, instead

of MoE, on the self-attention layer to further improve the performance of MIXLORA. Following the previous work [34], we also employ an auxiliary load balance loss for MIXLORA to address the expert imbalance problem. Furthermore, we design a high-performance framework to optimize the computation process of multiple LoRA based experts in MIXLORA both for training and inference[1]. As a result, this framework reduces the computational complexity of MIXLORA by 30%, and saves about 40% GPU memory usage when training or inferencing multiple MIXLORA models.

We validate the effectiveness of MIXLORA across a wide variety of tasks. For baselines, we choose the widely used PEFT method LoRA [15] and the current state-of-the-art PEFT method DoRA [19]. Quantitative results demonstrate that MIXLORA consistently outperforms LoRA and DoRA in both single-task and multi-task learning scenarios. For single-task learning, MIXLORA achieves an average accuracy improvement of **5.8%** compared to LoRA and **2.3%** compared to DoRA on LLaMA-2 7B. In multi-task learning, MIXLORA significantly surpasses LoRA by **9.8%** and DoRA by **9%** in accuracy, while demonstrating less performance degradation compared to the baseline methods. In summary, our contributions in this paper are as follows:

1. We introduce MIXLORA, a parameter-efficient mixture-of-experts method that constructs multiple LoRA based experts and a frozen shared FFN block from a pre-trained dense model. This approach simplifies the creation of an efficient sparse mixture-of-experts model with limited resources, and enhances performance across various downstream tasks.

2. We implement a high-throughput framework for the training and inference process. By optimizing redundant overhead in the computation process, we achieve approximately a **30%** reduction in the token computation latency of MIXLORA, and save **40%** or more in GPU memory usage during both training and inference of multiple MIXLORA models on a single consumer-grade GPU with 24GB memory, using LLaMA2-7B in half precision.

3. We conduct comprehensive evaluations on several commonly used benchmarks, including ARC [35], BoolQ [36], OpenBookQA [37], PIQA [38], SIQA [39], HellaSwag [40] and WinoGrande [41]. The results demonstrate that MIXLORA exhibits superior performance in handling various downstream tasks compared to existing fine-tuning methods. Specifically, MIXLORA increase the average accuracy by **5.8%** on single-task evaluations and **9.8%** on multi-task evaluations for LoRA with LLaMA2-7B.

2 Related Works

PEFT For LLMs. Large Language Models (LLMs)[1; 2; 3; 4; 5] have demonstrated remarkable capabilities in NLP tasks. Following this advancement, instruction finetuning [6; 7; 8] has further enabled LLMs to understand human intentions and follow instructions, serving as the foundation of chat systems [42; 43]. However, as the size of LLMs scales up, finetuning them becomes a process that is time-consuming, and memory-intensive. To mitigate this issue, various studies explore different approaches: parameter-efficient finetuning (PEFT)[44], distillation[45; 46], quantization [47; 48], pruning [49; 50], etc. LoRA [15], leveraging low-rank matrices to decompose linear layer weights is one of the most popular PEFT methods, thus enhancing model performance without introducing any additional computational overhead during inference. For instance, VeRA [51] incorporate learnable scaling vectors to adjust shared pairs of frozen random matrices across layers. Furthermore, FedPara [52] concentrates on low-rank Hadamard products for federated learning scenarios. Tied-Lora [53] implement weight tying to further reduce the number of trainable parameters. AdaLoRA [54] employ Singular Value Decomposition (SVD) to decompose matrices and prune less significant singular values for streamlined updates. DoRA [19] decomposes pre-trained weights into two components, magnitude and direction, and utilizes LoRA for directional updates during fine-tuning, thereby efficiently reducing the number of trainable parameters.

Mixture-of-Experts. The concept of Mixture-of-Experts (MoE) [55] dates back to as early as 1991, introducing a novel supervised learning approach involving multiple networks (experts), each specialized in handling a subset of training examples. The modern incarnation of MoE modifies the traditional feed-forward sub-layer within transformer blocks by incorporating sparsely activated experts, thereby enabling substantial increases in model width without a corresponding surge in computational demands. Various MoE architectures have emerged, distinguished by their sampling

[1] GitHub: `https://github.com/TUDB-Labs/MixLoRA`

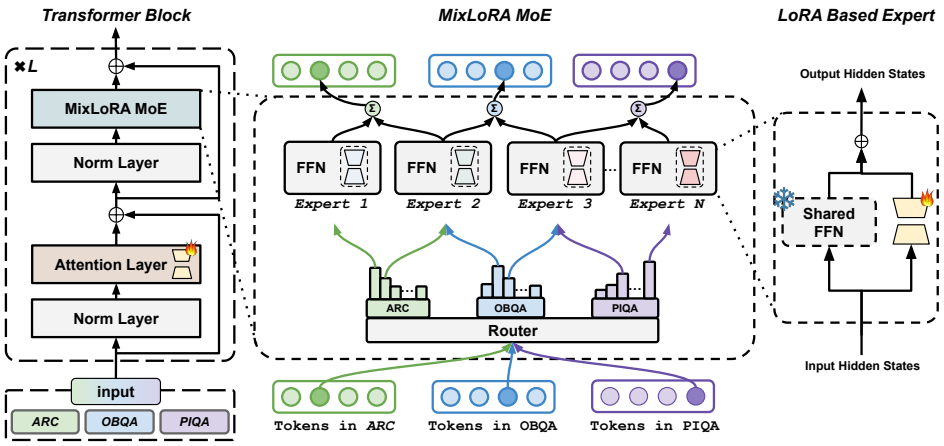


Figure 2: The architecture of MIXLORA transformer block. MIXLORA consists of n experts formed by an original FFN sublayer combined with different LoRAs, where the weights of the FFN sublayer are shared among all experts.

strategies and routing mechanisms. Building on this evolution, LLaVA-MoLE [56] effectively routes tokens to domain-specific experts within transformer layers, mitigating data conflicts and achieving consistent performance gains over plain LoRA baselines. For other MoE based methods, MoRAL [23]addresses the challenge of adapting LLMs to new domains/tasks and enabling them to be efficient lifelong learners. LoRAMoE [26] integrates LoRAs using a router network to alleviate world knowledge forgetting. PESC [25] transitions dense models to sparse models using a MoE architecture, reducing computational costs and GPU memory requirements. MoE-LoRA [24] propose a novel parameter-efficient MoE method with Layer-wise Expert Allocation (MoLA) for Transformer-based models. MoCLE [27] proposes a MoE architecture to activate task-customized model parameters based on instruction clusters. As for ours, we integrate LoRAs as stochastic experts, reducing computational cost while expanding model capacity and enhancing LLMs' generalization ability.

3 MIXLORA

In this section, we first introduce the foundational concepts of LoRA and MoEs in Section 3.1 and the architectural design of MIXLORA in Section 3.2. Then, we explain how we optimize the performance of MIXLORA for model training and inference in Section 3.3,.

3.1 Preliminaries

LoRA. LoRA only tunes the additional adaptation parameters and replaces the original weight update. A LoRA block consists of two matrices, $\mathbf{B} \in \mathbb{R}^{d_1 \times r}$ and $\mathbf{A} \in \mathbb{R}^{r \times d_2}$, where d_1 and d_2 denote the dimensions of the pretrained weights $\mathbf{W}$ of the LLMs ($\mathbf{W} \in \mathbb{R}^{d_1 \times d_2}$). The parameter r denotes the hidden dimension of Lora, with $r \ll min(d_1, d_2)$. Then, the updated weights $\mathbf{W}^{'}$ are calculated through:

$$\boldsymbol{W}' = \boldsymbol{W} + \boldsymbol{BA} \tag{1}$$

Mixture-of-Experts (MoE). The MoE architecture is initially introduced to language models through GShard [57]. An MoE layer consists of n experts, denoted by $\{E_i\}_{i=1}^{n}$, along with a router R. The output $\mathbf{h}^{'}$ of an MoE layer for a given hidden states $\mathbf{h}$ is determined by:

$$\mathbf{h}^{'} = \sum_{i=1}^{n} R(\mathbf{h})_i E_i(\mathbf{h}). \tag{2}$$

Here, $R(\mathbf{h})_i$ indicates the router's output for the i-th expert for selecting specific experts, and $E_i(\mathbf{h})$ is the result from the i-th expert.

3.2 Architecture of MIXLORA

As shown in Figure 2, MIXLORA is constructed from two main parts. The first part involves constructing the sparse MoE block using the vanilla transformer block augmented with LoRAs. The second part utilizes a top-k router to assign each token from various tasks (such as ARC, OBQA, PIQA, etc.) to different expert modules. Given the input text is $\boldsymbol{s} = (s_1, s_2, \ldots, s_n)$ with label of $\boldsymbol{y}$. Let $\mathbf{h}_i^\ell \in \mathbb{R}^{1\times d}$ $(1 \le i \le n, 1 \le \ell \le L)$ denote the output hidden state of the i-th token at the ℓ-th large language model (LLM) layer, where L is the total number of the LLM layers and d is the hidden dimension. The large language model consists of stacked multi-head self-attention (MSA) and feed-forward neural networks (FFN). Layer normalization (LN) and residual connections are applied within each block [58; 59]. Formally, the output $\mathbf{h}^\ell$ of the ℓ-th LLM layer in a normal transformers block is calculated via:

$$\mathbf{h}^0 = [s_1, s_2, \cdots, s_n], \tag{3}$$

$$\mathbf{z}^\ell = \text{MSA}(\text{LN}(\mathbf{h}^{\ell-1})) + \mathbf{h}^{\ell-1}, \quad \mathbf{h}^\ell = \text{FFN}(\text{LN}(\mathbf{z}^\ell)) + \mathbf{z}^\ell \tag{4}$$

MIXLORA Forward. MIXLORA constructs experts based on the LoRA technique [15]. MIXLORA utilizes LoRA to effectively store the updated parameters of each expert during fine-tuning, rather than employing LoRA solely for constructing each expert. This approach aligns MIXLORA more closely with existing pre-trained MoE models. In MIXLORA's MoE blocks, the base weights of these experts are shared from a single feed-forward network (FFN) of the dense model (as illustrated in Figure 2) to enhance both training and inference efficiency:

$$\mathbf{h}^\ell = \text{MixLoRA}(\text{LN}(\mathbf{z}^\ell)) + \mathbf{z}^\ell \tag{5}$$

$$\text{MixLoRA}(\mathbf{h}^\ell) = \sum_{k=1}^{K} R^\ell(\mathbf{h}^\ell)_k E_k^\ell(\mathbf{h}^\ell), \quad E_k^\ell(\mathbf{h}^\ell) = \boldsymbol{W}^\ell \cdot \mathbf{h}^\ell + \boldsymbol{B}_i^\ell \boldsymbol{A}_i^\ell \cdot \mathbf{h}^\ell \tag{6}$$

where $\boldsymbol{W}$ is the pretrained weights of the FFN layer, which is shared by $\{E_k\}_{k=1}^K$, $R(\cdot)$ denotes the Top-K router we employed to select specific LoRA experts for different tokens and tasks and $E_k(\cdot)$ represents k-th LoRA experts in the MIXLORA module. The role of MIXLORA is to replace the FFN layer of the dense models in Equation 5, and its key concept is to select different experts by the router for each token, where each expert is composed of a different LoRA and origin FFN layer (Equation 6).

Top-K Router. The Top-K router in an MoE layer determines the assignment of each token to the most suitable experts [57]. The router is a linear layer that computes the probability of the input token $\mathbf{x}_i$ being routed to each expert: $\boldsymbol{W_r}(s)$. Within the sparse transformer block, this router activates the most appropriate LoRA experts based on the input tokens. It leverages the softmax activation function to model a probability distribution over the experts. The router's weights $\boldsymbol{W}_r$ are the trainable parameters of the routing network. As shown in Figure 2, a top-2 gate router is employed in our design, which chooses the best two experts from n available $\{E_k\}_{k=1}^K$ for each input token $\mathbf{x}_i$:

$$R^\ell(\mathbf{h}_i^\ell) = \text{KeepTop-2}(\text{Softmax}(\boldsymbol{W}_r^\ell \cdot \mathbf{x}_i)). \tag{7}$$

During inference, the top-k gate router dynamically selects the best k experts for each token. Through this mechanism, the mix-experts and the router work in tandem, enabling the experts to develop varied capacities and efficiently handle diverse types of tasks.

Experts Load Balance. Unbalanced load of experts is a significant challenge for MoEs. This is because some experts tend to be chosen more frequently by the top-k routers [34]. To encounter this load imbalance, we apply load balancing loss to mitigate the unbalanced load for experts when training. Inspired by Switch Transformers [60], we calculate the auxiliary loss and add it to a total loss. Given N experts indexed by $i = 1$ to N and a batch B with T tokens, the auxiliary loss is computed as following:

$$\mathcal{L}_{\text{aux}} = a \cdot N \cdot \sum_{i=1}^{N} \mathcal{F}_i \cdot \mathcal{P}_i, \tag{8}$$

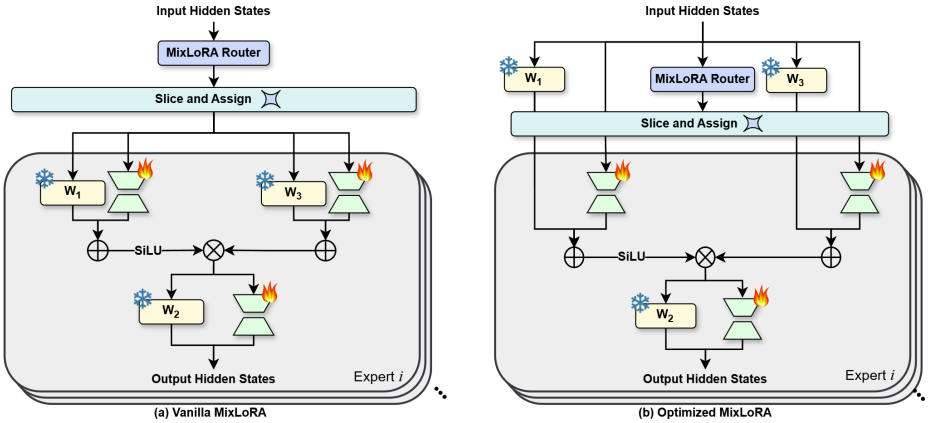


Figure 3: Comparison of the forward propagation processes: (a) the process in a vanilla MIXLORA MoE block; (b) the optimized process that shares computation results of W_1 and W_3 to reduce computational complexity.

$$\mathcal{F}_i = \frac{1}{T} \sum_{x \in B} \mathbb{1}\{\text{argmax}_k \, R(x)_k = i\}, \mathcal{P}_i = \frac{1}{T} \sum_{x \in B} R(x)_i. \tag{9}$$

Where $R(\cdot)$ is the top-k router, $\mathcal{F}_i$ is the fraction of tokens dispatched to expert i and $\mathcal{P}_i$ is the fraction of the router probability allocated for expert i. The final loss is multiplied by the expert count N to keep the loss constant as the number of experts varies. Additionally, we utilize $a = 10^{-2}$ as a multiplicative coefficient for auxiliary losses, which is large enough to ensure load balancing while remaining small enough not to overwhelm the primary cross-entropy objective.

Adding LoRA with Attention Layer. MIXLORA further extends its fine-tuning capabilities to encompass the attention layer. Previous studies, such as ST-MoE [32], have suggested that fine-tuning the attention layer can significantly improve performance. To enhance the fine-tuning process with MIXLORA, we integrate LoRA adapters into the attention layer of the dense model (as shown in Figure 2). Experimental results demonstrate that the MIXLORA model, fine-tuned with q, k, v, and o projection, consistently achieves superior average scores compared to the identical configuration trained solely with a sparse layer of mixture of LoRA experts (i.e., MIXLORA MoE).

3.3 Performance Optimization of MIXLORA

Reducing the Computational Complexity (I). As describes in previous section, each expert network in MIXLORA includes a shared frozen FFN and several LoRAs used for storing the updated parameters of each linear projection layer of experts during fine-tuning. This setup causes the computational complexity to vary based on the K settings of the router. This is because the token-level Top-K router in MIXLORA sends each token to K experts for computation and then aggregates their residuals to produce the output. Taking LLaMA as an example, the FFN block of LLaMA consists of three linear projection weights: W_1, W_2, and W_3, and the forward propagation process can be expressed as $H = W_2(\text{SiLU}(W_1(x)) \cdot W_3(x))$. As shown in Figure 3 (a), each expert in MIXLORA has the same forward propagation process, except that every linear projection layer has a separate LoRA, and the input x of each expert is pre-allocated by the MIXLORA router. This introduces a notable overhead when processing long sequence inputs, posing a significant challenge for the performance optimization of MIXLORA: *How to reduce the computational complexity of* MIXLORA *while maintaining model accuracy?*

Considering we have shared the weights of the FFN block, we further share the computation results to reduce computational complexity. As shown in Figure 3 (b), rather than pre-allocating the input sequence of the MIXLORA block, the efficient approach first sends the input directly to W_1 and W_3 of the FFN block in parallel, then slices the output of these linear projections by the routing weights output by the MIXLORA router. The computational complexity of the W_2 projection cannot be reduced because its computation process depends on the outputs of the W_1 and W_3 projections. This approach significantly reduces one-third of the computational complexity of the MIXLORA

Table 1: Comparison of different PEFT methods for single-task learning, using base models with different architectures and number of parameters. Reported results are accuracy scores.

Model	Method	# Params	ARC-e	ARC-c	BoolQ	OBQA	PIQA	SIQA	HellaS	WinoG	AVG.
Gemma 2B	LoRA	3.2%	71.9	43.2	62.1	71.4	80.9	71.4	84.4	46.8	66.5
	DoRA	3.2%	71.5	46.2	62.2	70.4	81.6	71.9	85.4	50.4	67.5
	MixLoRA	4.3%	76.3	47.4	65.8	75.8	81.1	73.6	89.0	50.4	**69.9**
	MixDoRA	4.3%	77.0	54.3	67.2	75.4	81.8	75.9	89.3	51.6	**71.6**
LLaMA-2 7B	LoRA	2.9%	73.8	50.9	62.2	80.4	82.1	69.9	88.4	66.8	71.8
	DoRA	2.9%	76.5	59.8	71.7	80.6	82.7	74.1	89.6	67.3	75.3
	MixLoRA	2.9%	77.7	58.1	72.7	81.6	83.2	78.0	93.1	76.8	**77.6**
	MixDoRA	2.9%	77.5	58.2	72.6	80.9	82.2	80.4	90.6	83.4	**78.2**
LLaMA-3 8B	LoRA	2.6%	89.0	75.7	67.2	85.0	80.7	78.3	74.2	75.3	78.2
	DoRA	2.6%	88.1	76.4	61.7	80.6	82.3	76.2	78.8	83.7	78.5
	MixLoRA	3.0%	86.5	79.9	75.0	84.8	87.6	78.8	93.3	82.1	**83.5**
	MixDoRA	3.0%	87.7	78.9	76.8	86.9	83.4	80.1	94.6	84.2	**84.1**
LLaMA-2 13B	LoRA	2.4%	83.2	67.6	75.4	83.2	86.7	80.0	94.3	81.9	81.5
	DoRA	2.4%	83.1	67.7	75.1	84.5	87.8	80.1	94.8	82.4	81.9
	MixLoRA	2.5%	83.5	69.9	77.1	83.0	86.8	82.5	94.7	86.3	**83.0**
	MixDoRA	2.5%	83.7	68.4	76.9	83.4	86.9	83.9	95.2	86.5	**83.1**

MoE block. In our experiments, the token computation latency of this approach was approximately 30% less than that of the vanilla MIXLORA, while maintaining the same model performance.

Optimizing Multi-MIXLORA Training and Inference (II). Inspired by the *Multi-LoRA Optimization* proposed by m-LoRA [33], we also optimized MIXLORA for multi-model high-throughput training and inference. Previously, we reduced the computational complexity of MIXLORA by eliminating duplicated calculations. When training and inferencing with two or more MIXLORA models, the multi-task inputs of these models are packed into a single batch to improve the training throughput. Specifically, we first send the batched inputs into W_1 and W_3 in parallel, and then slice the outputs of these linear projections using the separate routing weights of different MIXLORA models. This approach significantly reduces the memory usage of multiple MIXLORA models by sharing the same pre-trained model weights. In our experiments, the peak GPU memory usage of this approach was reduced by approximately 45% per model, while maintaining the same token computation latency. We describe our optimization algorithm in detail in Appendix A.7.

4 Experiments

4.1 Experimental Setup

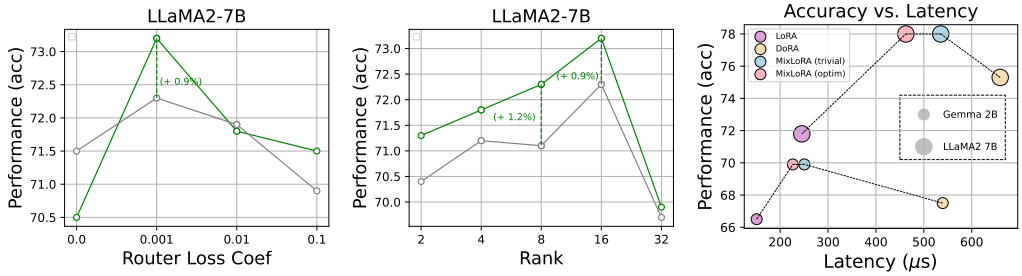
Datasets. To assess MIXLORA, we conducted experiments on diverse commonsense reasoning datasets, including question-answer tasks (ARC [35], OpenBookQA [37], PIQA [61], SocialIQA [39]), a classification task (BoolQ [36]), a science completion task (Hellaswag [40]), and a fill-in-the-blank task (Winogrande [41]). These datasets evaluate LLMs on various challenges from scientific queries to commonsense inference. The performance of all methods was measured using the accuracy metric across all datasets. Details can be found in Appendix A.2.

Baselines. We employ commonly utilized language models: LLaMA-2 7B, LLaMA-2 13B, along with the recently introduced Gemma 2B, and LLaMA-3 8B for LoRA and DoRA, correspondingly. To maintain uniformity in parameter sizes across all PEFT methods, we instantiate LoRA and DoRA with $r = 80$ and activate q, k, v, o in attention layers, as well as w_1, w_2, w_3 in feed-forward layers, serving as our baseline techniques. Detailed hyperparameters can be found in Appendix A.1.

Settings. In order to comprehensively evaluate the effectiveness of our method, we apply it on the basis of LoRA and DoRA, which are labeled as **MIXLORA** and **MIXDORA** respectively in the experiment. Both MIXLORA and MIXDORA are configured with $r = 16$, incorporating 8 experts and a top-2 router mechanism. We apply the q, k, v, o parameters on the attention layers, as well as the weights w_1, w_2, w_3 on the feed-forward layers for the experts.

Table 2: Comparison of different PEFT methods for multi-task learning, using LLaMA2 7B as the base model. Single-Task (**ST**) setup refers to training and evaluating PEFT modules for each task, while Multi-Task (**MT**) setup refers to training on mixed tasks, followed by separate evaluation. Reported results are accuracy scores.

PEFT Method	# Params (%)	ST/MT	ARC-e	ARC-c	BoolQ	OBQA	PIQA	AVG.
LoRA	2.9%	ST	73.8	50.9	62.2	80.4	82.1	69.9
	2.9%	**MT**	61.3	55.7	66.7	71.6	72.4	65.5
			-12.5	4.8	4.5	-8.8	2.5	-1.9
DoRA	2.9%	ST	76.5	59.8	71.7	80.6	82.7	74.3
	2.9%	**MT**	64.5	54.1	65.4	75.8	71.9	66.3
			-12.0	-5.7	-6.3	-2.8	-6.9	-6.7
MixLoRA	2.9%	ST	77.7	58.1	72.7	81.6	83.2	74.7
	2.9%	**MT**	76.6	64.2	71.2	81.6	82.7	75.3
			-1.1	6.1	-1.5	-	-0.5	0.6
MixDoRA	2.9%	ST	77.5	58.2	72.6	80.9	82.2	74.3
	2.9%	**MT**	76.9	63.4	71.8	82.2	80.4	74.9
			-0.6	5.2	0.8	1.3	-1.8	0.6



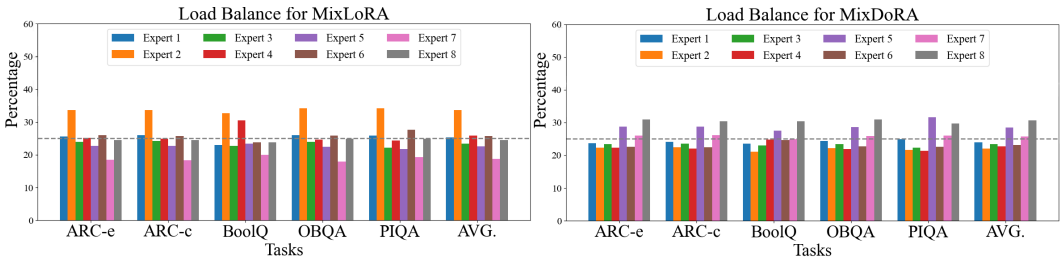
(a) (b) (c)

Figure 4: Ablation studies on router loss coefficient (a) and rank (b) on LLaMA2 7B. (c) MIXLORA outperforms LoRA and DoRA without introducing significant latency on diverse commonsense tasks.

4.2 Main Results

Single-Task Setup. Table 1 compares the performance of LoRA, DoRA, MIXLORA, and MIXDORA when employing these methods for fine-tuning on a single evaluation task. The results demonstrate that DoRA outperforms LoRA in most evaluations, and our methods **MIXLORA** and **MIXDORA** achieve commendable performance across all evaluation metrics. However, MIXDORA does not consistently exhibit superior performance compared to MIXLORA, only showing a small advantage in average accuracy. Additionally, in most evaluations, it can be observed that the fine-tuning results of MIXLORA and MIXDORA on LLaMA3 8B (83.5%, 84.1%) surpass the results on LLaMA2 13B (83.0%, 83.1%), while LoRA and DoRA leave a significant gap (8B: 78.2%, 78.5%; 13B: 81.5%, 81.9%). This indicates that our method effectively extends the model's capacity by building multiple experts on the FFN of the dense model.

Multi-Task Setup. Table 2 presents the results of LoRA, DoRA, MIXLORA, and MIXDORA with LLaMA2-7B in multi-task learning. In contrast to the single-task learning results shown in Table 1, during multi-task learning, we mixed training data from ARC, BoolQ, OBQA, and PIQA to train the model, followed by separate evaluations to investigate the generalization ability of each method. The results indicate that, compared to single-task learning, LoRA and DoRA exhibit degradation in average accuracy in multi-task learning (LoRA: -1.9%, DoRA: -6.7%), while MIXLORA and MIXDORA maintain nearly the same average accuracy. Additionally, we conducted multi-task learning on Gemma 2B, with results presented in Appendix A.3. MIXLORA and MIXDORA maintain their superiority, showing the lowest performance degradation. This suggests that MIXLORA and MIXDORA demonstrate stronger generalization ability and mitigate the issue of knowledge forgetting in multi-task learning compared to LoRA and DoRA.



(a) (b)

Figure 5: Distribution of expert loadings. The average workload of the 8 experts in MIXLORA (a) and MIXDORA (b) during the evaluation of multi-task learning is depicted in the figure. The average standard deviation of MIXLORA is smaller than that of MIXDORA (0.0223 < 0.0328). However, both standard deviations are small enough, indicating that the workload of these experts is balanced.

4.3 Ablation Study

Analysis of Auxiliary Loss. We investigated the influence of different router loss coefficients on model performance with ARC, OpenBookQA, and BoolQ using LLaMA2-7B. As shown in Figure 4 (a), results indicate that with the coefficient of 1e-3, MIXLORA achieves the highest average accuracy. Disabling router loss or using a higher coefficient results in lower average accuracy. This suggests that a reasonable router loss coefficient can help address the imbalance problem of experts, while a higher coefficient can impede model convergence during fine-tuning. Furthermore, MIXLORA imposes higher requirements on the rationality of the router loss coefficient compared to MIXDORA, indicating that MixDoRA is less sensitive to hyperparameter settings than MIXLORA. For example, when disabling router loss (coefficient = 0), MIXLORA lags behind MIXDORA by 1% in average accuracy, while surpassing MIXDORA by 0.9% in average accuracy when the coefficient is set to 1e-3.

To confirm the effectiveness of auxiliary loss, we also present the distribution of experts across diverse tasks for MIXLORA and MIXDORA in Figure 5. Intuitively, we can observe that the expert loads in all tasks are balanced, indicating that these experts are allocated to different tasks evenly. Specifically, both MIXLORA and MIXDORA achieves low average standard deviation (0.0223 and 0.0328). This verifies that the use of auxiliary load balance loss mitigates the imbalance problem of experts.

Analysis of Model Rank. We investigated the influence of different ranks on model performance with ARC, OpenBookQA, and BoolQ using LLaMA2-7B. As shown in Figure 4 (b), both MIXLORA and MIXDORA consistently perform well from rank 2 to rank 16. However, the average accuracy of them both drops when rank is 32 due to convergence difficulties. Moreover, MIXDORA slightly lags behind MIXLORA at all ranks. This is because our method introduces more fine-tuning diversity through the MoE structure, making DoRA's weight decomposition method less effective when applied to our method (MIXDORA). For example, in our previous multi-task experiments on Tabel 2, DoRA often showed more degradation compared to MIXLORA and even LoRA.

Analysis of Computation Efficiency. To measure the computational overhead between LoRA, DoRA, and MIXLORA, we collected the token computation latency and peak GPU memory usage from PEFT for LoRA and DoRA, and from m-LoRA for MIXLORA during training and inference separately. The trainable parameters for all methods were strictly controlled to be equal both on Gemma 2B and LLaMA2 7B. Figure 4 (c) shows the overall comparison of model accuracy and token computation latency between different model sizes (2B and 7B). Our method, MIXLORA, achieves the best accuracy in both Gemma 2B and LLaMA2 7B, while the token computation latency falls between LoRA and DoRA. The optimized MIXLORA (in Section 3.3) shows significant improvement while maintaining the same model accuracy, especially with the larger 7B model. Details can be found in Appendix A.4.

5 Conclusion

In this paper, we introduce MIXLORA, a parameter-efficient MoE method using multiple LoRA based experts and a frozen shared FFN block. Unlike traditional LoRA-MoE approaches, MIXLORA fused multiple LoRAs with the shared FFN layer and employs them to store updated parameters for each expert during fine-tuning, aligning it more with pre-trained MoE models. It also employs self-attention LoRA adapters and an auxiliary load balance loss to improve performance and address router imbalance. Furthermore, we design a high-performance framework to optimize the computation process of multiple LoRA based experts in MIXLORA both for training and inference. As a result, this framework reduces the computational complexity of MIXLORA by 30%, and saves about 40% GPU memory usage when training or inferencing multiple MIXLORA models. Evaluation shows MIXLORA outperforms baselines in single-task and multi-task scenarios. For single-task learning, MIXLORA achieves an average accuracy improvement of **5.8%** on LLaMA-2 7B compared to LoRA and **2.3%** compared to DoRA. In multi-task learning, MIXLORA significantly surpasses LoRA by **9.8%** and DoRA by **9%** in accuracy.

References

[1] Tom B. Brown, Benjamin Mann, Nick Ryder, Melanie Subbiah, Jared Kaplan, Prafulla Dhariwal, Arvind Neelakantan, Pranav Shyam, Girish Sastry, Amanda Askell, Sandhini Agarwal, Ariel Herbert-Voss, Gretchen Krueger, T. J. Henighan, Rewon Child, Aditya Ramesh, Daniel M. Ziegler, Jeff Wu, Clemens Winter, Christopher Hesse, Mark Chen, Eric Sigler, Mateusz Litwin, Scott Gray, Benjamin Chess, Jack Clark, Christopher Berner, Sam McCandlish, Alec Radford, Ilya Sutskever, and Dario Amodei. Language models are few-shot learners. ArXiv, abs/2005.14165, 2020.

[2] Aakanksha Chowdhery, Sharan Narang, Jacob Devlin, Maarten Bosma, Gaurav Mishra, Adam Roberts, Paul Barham, Hyung Won Chung, Charles Sutton, Sebastian Gehrmann, Parker Schuh, Kensen Shi, Sasha Tsvyashchenko, Joshua Maynez, Abhishek Rao, Parker Barnes, Yi Tay, Noam M. Shazeer, Vinodkumar Prabhakaran, Emily Reif, Nan Du, Benton C. Hutchinson, Reiner Pope, James Bradbury, Jacob Austin, Michael Isard, Guy Gur-Ari, Pengcheng Yin, Toju Duke, Anselm Levskaya, Sanjay Ghemawat, Sunipa Dev, Henryk Michalewski, Xavier García, Vedant Misra, Kevin Robinson, Liam Fedus, Denny Zhou, Daphne Ippolito, David Luan, Hyeontaek Lim, Barret Zoph, Alexander Spiridonov, Ryan Sepassi, David Dohan, Shivani Agrawal, Mark Omernick, Andrew M. Dai, Thanumalayan Sankaranarayana Pillai, Marie Pellat, Aitor Lewkowycz, Erica Moreira, Rewon Child, Oleksandr Polozov, Katherine Lee, Zongwei Zhou, Xuezhi Wang, Brennan Saeta, Mark Díaz, Orhan Firat, Michele Catasta, Jason Wei, Kathleen S. Meier-Hellstern, Douglas Eck, Jeff Dean, Slav Petrov, and Noah Fiedel. Palm: Scaling language modeling with pathways. J. Mach. Learn. Res., 24, 2022.

[3] Jordan Hoffmann, Sebastian Borgeaud, Arthur Mensch, Elena Buchatskaya, Trevor Cai, Eliza Rutherford, Diego de Las Casas, Lisa Anne Hendricks, Johannes Welbl, Aidan Clark, Tom Hennigan, Eric Noland, Katie Millican, George van den Driessche, Bogdan Damoc, Aurelia Guy, Simon Osindero, Karen Simonyan, Erich Elsen, Jack W. Rae, Oriol Vinyals, and L. Sifre. Training compute-optimal large language models. ArXiv, abs/2203.15556, 2022.

[4] Hugo Touvron, Thibaut Lavril, Gautier Izacard, Xavier Martinet, Marie-Anne Lachaux, Timothée Lacroix, Baptiste Rozière, Naman Goyal, Eric Hambro, Faisal Azhar, Aurelien Rodriguez, Armand Joulin, Edouard Grave, and Guillaume Lample. Llama: Open and efficient foundation language models. ArXiv, abs/2302.13971, 2023.

[5] Hugo Touvron, Louis Martin, Kevin R. Stone, Peter Albert, Amjad Almahairi, Yasmine Babaei, Nikolay Bashlykov, Soumya Batra, Prajjwal Bhargava, Shruti Bhosale, Daniel M. Bikel, Lukas Blecher, Cristian Cantón Ferrer, Moya Chen, Guillem Cucurull, David Esiobu, Jude Fernandes, Jeremy Fu, Wenyin Fu, Brian Fuller, Cynthia Gao, Vedanuj Goswami, Naman Goyal, Anthony S. Hartshorn, Saghar Hosseini, Rui Hou, Hakan Inan, Marcin Kardas, Viktor Kerkez, Madian Khabsa, Isabel M. Kloumann, A. V. Korenev, Punit Singh Koura, Marie-Anne Lachaux, Thibaut Lavril, Jenya Lee, Diana Liskovich, Yinghai Lu, Yuning Mao, Xavier Martinet, Todor Mihaylov, Pushkar Mishra, Igor Molybog, Yixin Nie, Andrew Poulton, Jeremy Reizenstein, Rashi Rungta, Kalyan Saladi, Alan Schelten, Ruan Silva, Eric Michael Smith, R. Subramanian, Xia Tan, Binh Tang, Ross Taylor, Adina Williams, Jian Xiang Kuan, Puxin Xu, Zhengxu Yan, Iliyan Zarov, Yuchen Zhang, Angela Fan, Melanie Kambadur, Sharan Narang, Aurelien Rodriguez, Robert Stojnic, Sergey Edunov, and Thomas Scialom. Llama 2: Open foundation and fine-tuned chat models. ArXiv, abs/2307.09288, 2023.

[6] Hyung Won Chung, Le Hou, S. Longpre, Barret Zoph, Yi Tay, William Fedus, Eric Li, Xuezhi Wang, Mostafa Dehghani, Siddhartha Brahma, Albert Webson, Shixiang Shane Gu, Zhuyun Dai, Mirac Suzgun, Xinyun Chen, Aakanksha Chowdhery, Dasha Valter, Sharan Narang, Gaurav Mishra, Adams Wei Yu, Vincent Zhao, Yanping Huang, Andrew M. Dai, Hongkun Yu, Slav Petrov, Ed Huai hsin Chi, Jeff Dean, Jacob Devlin, Adam Roberts, Denny Zhou, Quoc V. Le, and Jason Wei. Scaling instruction-finetuned language models. ArXiv, abs/2210.11416, 2022.

[7] Srinivas Iyer, Xi Victoria Lin, Ramakanth Pasunuru, Todor Mihaylov, Daniel Simig, Ping Yu, Kurt Shuster, Tianlu Wang, Qing Liu, Punit Singh Koura, Xian Li, Brian O'Horo, Gabriel Pereyra, Jeff Wang, Christopher Dewan, Asli Celikyilmaz, Luke Zettlemoyer, and Veselin Stoyanov. Opt-iml: Scaling language model instruction meta learning through the lens of generalization. ArXiv, abs/2212.12017, 2022.

[8] Lianmin Zheng, Wei-Lin Chiang, Ying Sheng, Siyuan Zhuang, Zhanghao Wu, Yonghao Zhuang, Zi Lin, Zhuohan Li, Dacheng Li, Eric Xing, et al. Judging llm-as-a-judge with mt-bench and chatbot arena. NeurIPS, 36, 2024.

[9] Jason Wei, Yi Tay, Rishi Bommasani, Colin Raffel, Barret Zoph, Sebastian Borgeaud, Dani Yogatama, Maarten Bosma, Denny Zhou, Donald Metzler, et al. Emergent abilities of large language models. arXiv preprint arXiv: 2206.07682, 2022.

[10] Neil Houlsby, Andrei Giurgiu, Stanislaw Jastrzebski, Bruna Morrone, Quentin De Laroussilhe, Andrea Gesmundo, Mona Attariyan, and Sylvain Gelly. Parameter-efficient transfer learning for nlp. In ICML, pages 2790–2799. PMLR, 2019.

[11] Xiang Lisa Li and Percy Liang. Prefix-tuning: Optimizing continuous prompts for generation. ACL, 2021.

[12] Brian Lester, Rami Al-Rfou, and Noah Constant. The power of scale for parameter-efficient prompt tuning. In EMNLP, 2021.

[13] Elad Ben-Zaken, Shauli Ravfogel, and Yoav Goldberg. Bitfit: Simple parameter-efficient fine-tuning for transformer-based masked language-models. ArXiv, 2021.

[14] Haokun Liu, Derek Tam, Mohammed Muqeeth, Jay Mohta, Tenghao Huang, Mohit Bansal, and Colin Raffel. Few-shot parameter-efficient fine-tuning is better and cheaper than in-context learning. ArXiv, 2022.

[15] J. Edward Hu, Yelong Shen, Phillip Wallis, Zeyuan Allen-Zhu, Yuanzhi Li, Shean Wang, and Weizhu Chen. Lora: Low-rank adaptation of large language models. ArXiv, abs/2106.09685, 2021.

[16] Zehua Sun, Huanqi Yang, Kai Liu, Zhimeng Yin, Zhenjiang Li, and Weitao Xu. Recent advances in lora: A comprehensive survey. ACM Transactions on Sensor Networks, 2022.

[17] Jothi Prasanna Shanmuga Sundaram, Wan Du, and Zhiwei Zhao. A survey on lora networking: Research problems, current solutions, and open issues. IEEE Communications Surveys & Tutorials, 2019.

[18] Soufiane Hayou, Nikhil Ghosh, and Bin Yu. Lora+: Efficient low rank adaptation of large models. arXiv preprint arXiv: 2402.12354, 2024.

[19] Shih-Yang Liu, Chien-Yi Wang, Hongxu Yin, Pavlo Molchanov, Yu-Chiang Frank Wang, Kwang-Ting Cheng, and Min-Hung Chen. Dora: Weight-decomposed low-rank adaptation. arXiv preprint arXiv: 2402.09353, 2024.

[20] Wenfeng Feng, Chuzhan Hao, Yuewei Zhang, Yu Han, and Hao Wang. Mixture-of-loras: An efficient multitask tuning for large language models. arXiv preprint arXiv: 2403.03432, 2024.

[21] Chengsong Huang, Qian Liu, Bill Yuchen Lin, Tianyu Pang, Chao Du, and Min Lin. Lorahub: Efficient cross-task generalization via dynamic lora composition. arXiv preprint arXiv: 2307.13269, 2023.

[22] Jonas Pfeiffer, Aishwarya Kamath, Andreas Rücklé, Kyunghyun Cho, and Iryna Gurevych. Adapterfusion: Non-destructive task composition for transfer learning. arXiv preprint arXiv: 2005.00247, 2020.

[23] Shu Yang, Muhammad Asif Ali, Cheng-Long Wang, Lijie Hu, and Di Wang. Moral: Moe augmented lora for llms' lifelong learning. arXiv preprint arXiv: 2402.11260, 2024.

[24] Tongxu Luo, Jiahe Lei, Fangyu Lei, Weihao Liu, Shizhu He, Jun Zhao, and Kang Liu. Moelora: Contrastive learning guided mixture of experts on parameter-efficient fine-tuning for large language models. arXiv preprint arXiv: 2402.12851, 2024.

[25] Haoyuan Wu, Haisheng Zheng, and Bei Yu. Parameter-efficient sparsity crafting from dense to mixture-of-experts for instruction tuning on general tasks. arXiv preprint arXiv: 2401.02731, 2024.

[26] Shihan Dou, Enyu Zhou, Yan Liu, Songyang Gao, Jun Zhao, Wei Shen, Yuhao Zhou, Zhiheng Xi, Xiao Wang, Xiaoran Fan, Shiliang Pu, Jiang Zhu, Rui Zheng, Tao Gui, Qi Zhang, and Xuanjing Huang. Loramoe: Alleviate world knowledge forgetting in large language models via moe-style plugin. arXiv, 2024.

[27] Yunhao Gou, Zhili Liu, Kai Chen, Lanqing Hong, Hang Xu, Aoxue Li, Dit-Yan Yeung, James T Kwok, and Yu Zhang. Mixture of cluster-conditional lora experts for vision-language instruction tuning. arXiv preprint arXiv: 2312.12379, 2023.

[28] Qidong Liu, Xian Wu, Xiangyu Zhao, Yuanshao Zhu, Derong Xu, Feng Tian, and Yefeng Zheng. Moelora: An moe-based parameter efficient fine-tuning method for multi-task medical applications. ArXiv: 2310.18339, 2023.

[29] Chongyang Gao, Kezhen Chen, Jinmeng Rao, Baochen Sun, Ruibo Liu, Daiyi Peng, Yawen Zhang, Xiaoyuan Guo, Jie Yang, and VS Subrahmanian. Higher layers need more lora experts. arXiv preprint arXiv: 2402.08562, 2024.

[30] Albert Q Jiang, Alexandre Sablayrolles, Antoine Roux, Arthur Mensch, Blanche Savary, Chris Bamford, Devendra Singh Chaplot, Diego de las Casas, Emma Bou Hanna, Florian Bressand, et al. Mixtral of experts. arXiv preprint arXiv: 2401.04088, 2024.

[31] Ashish Vaswani, Noam Shazeer, Niki Parmar, Jakob Uszkoreit, Llion Jones, Aidan N Gomez, Łukasz Kaiser, and Illia Polosukhin. Attention is all you need. NeurIPS, 2017.

[32] Barret Zoph, Irwan Bello, Sameer Kumar, Nan Du, Yanping Huang, Jeff Dean, Noam Shazeer, and William Fedus. St-moe: Designing stable and transferable sparse expert models. arXiv preprint arXiv: 2202.08906, 2022.

[33] Zhengmao Ye, Dengchun Li, Jingqi Tian, Tingfeng Lan, Jie Zuo, Lei Duan, Hui Lu, Yexi Jiang, Jian Sha, Ke Zhang, and Mingjie Tang. Aspen: High-throughput lora fine-tuning of large language models with a single gpu. arXiv: 2312.02515, 2023.

[34] William Fedus, Barret Zoph, and Noam Shazeer. Switch transformers: Scaling to trillion parameter models with simple and efficient sparsity. JMLR, 2022.

[35] Peter Clark, Isaac Cowhey, Oren Etzioni, Tushar Khot, Ashish Sabharwal, Carissa Schoenick, and Oyvind Tafjord. Think you have solved question answering? try arc, the ai2 reasoning challenge. arXiv preprint arXiv: 1803.05457, 2018.

[36] Christopher Clark, Kenton Lee, Ming-Wei Chang, Tom Kwiatkowski, Michael Collins, and Kristina Toutanova. Boolq: Exploring the surprising difficulty of natural yes/no questions. arXiv preprint arXiv: 1905.10044, 2019.

[37] Todor Mihaylov, Peter Clark, Tushar Khot, and Ashish Sabharwal. Can a suit of armor conduct electricity? a new dataset for open book question answering. arXiv preprint arXiv: 1809.02789, 2018.

[38] Yonatan Bisk, Rowan Zellers, Jianfeng Gao, Yejin Choi, et al. Piqa: Reasoning about physical commonsense in natural language. AAAI, 2020.

[39] Maarten Sap, Hannah Rashkin, Derek Chen, Ronan LeBras, and Yejin Choi. Socialiqa: Commonsense reasoning about social interactions. arXiv preprint arXiv: 1904.09728, 2019.

[40] Rowan Zellers, Ari Holtzman, Yonatan Bisk, Ali Farhadi, and Yejin Choi. Hellaswag: Can a machine really finish your sentence? arXiv preprint arXiv: 1905.07830, 2019.

[41] Keisuke Sakaguchi, Ronan Le Bras, Chandra Bhagavatula, and Yejin Choi. Winogrande: An adversarial winograd schema challenge at scale. Communications of the ACM, 2021.

[42] OpenAI. Chatgpt: Optimizing language models for dialogue., 2022.

[43] Josh Achiam, Steven Adler, Sandhini Agarwal, Lama Ahmad, Ilge Akkaya, Florencia Leoni Aleman, Diogo Almeida, Janko Altenschmidt, Sam Altman, Shyamal Anadkat, et al. Gpt-4 technical report. arXiv preprint arXiv: 2303.08774, 2023.

[44] Sourab Mangrulkar, Sylvain Gugger, Lysandre Debut, Younes Belkada, Sayak Paul, and Benjamin Bossan. Peft: State-of-the-art parameter-efficient fine-tuning methods. `https://github.com/huggingface/peft`, 2022.

[45] Zechun Liu, Barlas Oğuz, Changsheng Zhao, Ernie Chang, Pierre Stock, Yashar Mehdad, Yangyang Shi, Raghuraman Krishnamoorthi, and Vikas Chandra. Llm-qat: Data-free quantization aware training for large language models. ArXiv, abs/2305.17888, 2023.

[46] Guangxuan Xiao, Ji Lin, and Song Han. Offsite-tuning: Transfer learning without full model. ArXiv, abs/2302.04870, 2023.

[47] Elias Frantar, Saleh Ashkboos, Torsten Hoefler, and Dan Alistarh. Gptq: Accurate post-training quantization for generative pre-trained transformers. ArXiv, abs/2210.17323, 2022.

[48] Guangxuan Xiao, Ji Lin, Mickael Seznec, Julien Demouth, and Song Han. Smoothquant: Accurate and efficient post-training quantization for large language models. ArXiv, abs/2211.10438, 2022.

[49] Elias Frantar and Dan Alistarh. Sparsegpt: Massive language models can be accurately pruned in one-shot. ArXiv, abs/2301.00774, 2023.

[50] Xinyin Ma, Gongfan Fang, and Xinchao Wang. Llm-pruner: On the structural pruning of large language models. ArXiv, abs/2305.11627, 2023.

[51] Dawid Jan Kopiczko, Tijmen Blankevoort, and Yuki Markus Asano. Vera: Vector-based random matrix adaptation. arXiv preprint arXiv: 2310.11454, 2023.

[52] Nam Hyeon-Woo, Moon Ye-Bin, and Tae-Hyun Oh. Fedpara: Low-rank hadamard product for communication-efficient federated learning. arXiv preprint arXiv: 2108.06098, 2021.

[53] Adithya Renduchintala, Tugrul Konuk, and Oleksii Kuchaiev. Tied-lora: Enhacing parameter efficiency of lora with weight tying. arXiv preprint arXiv: 2311.09578, 2023.

[54] Qingru Zhang, Minshuo Chen, Alexander Bukharin, Pengcheng He, Yu Cheng, Weizhu Chen, and Tuo Zhao. Adaptive budget allocation for parameter-efficient fine-tuning. In ICLR, 2023.

[55] Robert A. Jacobs, Michael I. Jordan, Steven J. Nowlan, and Geoffrey E. Hinton. Adaptive mixtures of local experts. Neural Computation, 1991.

[56] Shaoxiang Chen, Zequn Jie, and Lin Ma. Llava-mole: Sparse mixture of lora experts for mitigating data conflicts in instruction finetuning mllms. arXiv preprint arXiv: 2401.16160, 2024.

[57] Dmitry Lepikhin, HyoukJoong Lee, Yuanzhong Xu, Dehao Chen, Orhan Firat, Yanping Huang, Maxim Krikun, Noam Shazeer, and Zhifeng Chen. Gshard: Scaling giant models with conditional computation and automatic sharding. In ICLR, 2020.

[58] Qiang Wang, Bei Li, Tong Xiao, Jingbo Zhu, Changliang Li, Derek F Wong, and Lidia S Chao. Learning deep transformer models for machine translation. arXiv preprint arXiv: 1906.01787, 2019.

[59] Alexei Baevski and Michael Auli. Adaptive input representations for neural language modeling. arXiv preprint arXiv: 1809.10853, 2018.

[60] Zhiyuan Zeng, Qipeng Guo, Zhaoye Fei, Zhangyue Yin, Yunhua Zhou, Linyang Li, Tianxiang Sun, Hang Yan, Dahua Lin, and Xipeng Qiu. Turn waste into worth: Rectifying top-k router of moe. arXiv preprint arXiv: 2402.12399, 2024.

[61] Yonatan Bisk, Rowan Zellers, Ronan Le Bras, Jianfeng Gao, and Yejin Choi. Piqa: Reasoning about physical commonsense in natural language. arXiv: 1911.11641, 2019.

A Appendix

A.1 Hyperparameters and Implementation Details

Table 3: Hyperparameter configurations of LoRA/DoRA and MixLoRA/MixDoRA for fine-tuning Gemma-2B, LLaMA2-7B/13B, and LLaMA3-8B on the commonsense reasoning tasks.

Hyperparameters	**LoRA/DoRA**	**MixLoRA/MixDoRA**
Cutoff Length	512	
Learning Rate	2e-4	
Optimizer	AdamW	
Batch size	16	
Accumulation Steps	8	
Dropout	0.05	
# Epochs	2	
Where	Q, K, V, O, Up, Down, Gate	
LoRA Rank r	80	16
LoRA Alpha α	160	32
# Experts	-	8
Top-K	-	2

All experiments are conducted with GPUs having 24GB memory (RTX 3090, RTX A5000, RTX 4090) for 7B models, GPUs having 48GB memory (RTX A6000) for 8B and 13B models, and setup with Python 3.10 and Ubuntu 22.04 on x86-64 CPUs.

A.2 Datasets

Table 4 presents detailed information about the datasets used in our experiments, including their task names, respective domains, the number of training and test sets, task types.

Table 4: Description of Datasets used in experiments.

Task Name	**Domain**	**# Train**	**# Test**	**Task Type**
BoolQ	Wikipedia	9,427	3,270	Text Classification
ARC-E	Natural Science	2,250	2,380	Question Answering
ARC-C	Natural Science	1,120	1,170	Question Answering
OpenBookQA	Science Facts	4,957	500	Question Answering
PIQA	Physical Interaction	16,100	1,840	Question Answering
SIQA	Social Interaction	33,410	1,954	Question Answering
HellaSwag	Video Caption	39,905	10,042	Sentence Completion
WinoGrande	Winograd Schemas	9,248	1,267	Fill in the Blank

All datasets are downloaded from HuggingFace using the DATASETS library in Python.

A.3 Multi-task Learning Evaluation Result using Gemma-2B

Table 5: Comparision of different peft methods for multi-task learning on various tasks, using Gemma 2B as the base model. Single-Task (**ST**) setup refers to training and evaluating PEFT modules for each task, while Multi-Task (**MT**) setup refers to training on mixed tasks, followed by separate evaluation. Reported results are accuracy scores.

PEFT Method	# Params (%)	ST/MT	ARC-e	ARC-c	BoolQ	OBQA	PIQA	AVG.
LoRA	3.2%	ST	71.9	43.2	62.1	71.4	80.9	65.9
	3.2%	**MT**	64.9	50.2	66.4	64.8	75.7	64.4
			-7.0	7.0	4.3	-6.6	-5.2	-1.5
DoRA	3.2%	ST	71.5	46.2	62.2	70.4	81.6	66.4
	3.2%	**MT**	63.7	50.8	61.6	61.0	81.1	63.6
			-7.8	4.6	-0.6	-9.4	-0.5	-2.8
MixLoRA	4.3%	ST	76.3	47.4	65.8	75.8	81.1	69.3
	4.3%	**MT**	70.3	55.5	66.6	70.0	78.7	68.2
			-6.0	8.1	0.8	-5.8	-2.4	-1.1
MixDoRA	4.3%	ST	77.0	54.3	67.2	75.4	81.8	71.1
	4.3%	**MT**	71.1	56.3	65.9	70.6	79.1	68.6
			-5.9	2.0	-1.3	-4.8	-2.7	-2.5

A.4 Experimental Results of Performance Metrics.

Table 6: Experimental results of LLaMA-2 7B for performance metrics. The latency shown in the table represents the token computation latency, and the memory indicates the peak GPU memory collected by the profiler. To accurately measure the performance of LoRA and DoRA during inference, we conducted the experiments with weights unmerged. † represents methods with MIXLORA optimization.

PEFT Method	*Training*						*Inference*			
	Forward		**Backward**		**Memory**		**Forward**		**Memory**	
	μs	%	μs	%	GB	%	μs	%	GB	%
LoRA	245.3	100.0%	552.3	100.0%	15.2	100.0%	241.4	100.0%	13.7	100.0%
DoRA	659.4	268.8%	1193.8	216.1%	15.6	102.4%	645.3	267.3%	13.7	100.0%
MixLoRA	535.2	218.2%	1187.5	215.0%	15.1	99.5%	532.8	220.7%	13.7	100.0%
†MixLoRA	462.5	188.5%	1097.6	198.7%	15.1	99.5%	442.2	183.2%	13.7	100.0%
MixLoRA ×2	533.9	217.7%	1185.5	214.6%	8.8	57.7%	523.8	217.0%	7.2	52.5%
†MixLoRA ×2	441.0	179.8%	1072.3	194.1%	8.8	57.7%	441.4	182.8%	7.2	52.5%

Table 6 shows the results on LLaMA2 7B, demonstrating that MIXLORA exhibits lower token computation latency (DoRA requires 659.4μs for forward propagation, while MIXLORA only needs 535.2μs) and comparable peak GPU memory usage to DoRA (approximately 15GB). However, MIXLORA shows nearly twice the token computation latency of LoRA (245.3μs). This increased latency is due to MIXLORA sending each token to two experts for computation (when $K = 2$). Nonetheless, with our optimized algorithm, we reduced the token computation latency by nearly 30% for a single model (from 535.2μs to 462.5μs) and decreased the peak GPU memory per model by almost 45% when training or inferring with two models simultaneously (from 15.1GB to 8.8GB during training, and from 13.7GB to 7.2GB during inference). Appendix 7 shows the results on Gemma 2B, corroborating these findings and proving that our algorithm maintains robustness across different model sizes. In conclusion, experiments show that MIXLORA offers a more balanced trade-off, providing higher performance with reduced latency compared to the current state-of-the-art method, DoRA.

Table 7: Experimental results of Gemma 2B for performance metrics. The latency shown in the table represents the token computation latency, and the memory indicates the peak GPU memory collected by the profiler. To accurately measure the performance of LoRA and DoRA during inference, we conducted the experiments with weights unmerged. † represents methods with MIXLORA optimization.

PEFT Method	*Training*						*Inference*			
	Forward		**Backward**		**Memory**		**Forward**		**Memory**	
	μs	%	μs	%	GB	%	μs	%	GB	%
LoRA	151.1	100.0%	308.2	100.0%	11.4	100.0%	152.0	100.0%	10.6	100.0%
DoRA	539.4	356.9%	919.9	298.5%	11.4	100.0%	533.4	350.9%	10.6	100.0%
MixLoRA	250.6	165.8%	527.2	171.1%	11.2	97.7%	245.1	161.2%	10.5	99.8%
†MixLoRA	226.5	149.9%	525.2	170.4%	11.2	97.7%	224.0	147.4%	10.5	99.8%
MixLoRA ×2	249.6	165.1%	524.0	170.1%	7.6	66.9%	243.1	160.6%	6.5	61.5%
†MixLoRA ×2	223.8	148.1%	523.7	169.9%	7.6	66.9%	221.2	145.6%	6.5	61.5%

A.5 Robustness of MIXLORA Towards Different Rank

Table 8: Accuracy comparison of MIXLORA and MIXDORA with varying ranks for LLaMA2-7B on the commonsense reasoning tasks.

PEFT Method	Rank r	# Params (%)	ARC-e	ARC-c	BoolQ	OBQA	Avg.
MixLoRA	2	0.38%	76.1	56.4	73.3	79.2	71.3
	4	0.74%	76.2	56.5	73.8	80.8	71.8
	8	1.46%	76.9	56.8	74.2	81.2	72.3
	16	2.91%	77.7	58.1	72.7	84.4	**73.2**
	32	5.80%	79.1	54.1	70.0	76.4	69.9
MixDoRA	2	0.38%	75.3	52.7	73.3	80.4	70.4
	4	0.74%	76.2	55.0	73.2	80.4	71.2
	8	1.46%	76.7	55.5	73.5	78.6	71.1
	16	2.91%	77.5	58.2	72.6	80.9	**72.3**
	32	5.80%	75.5	53.6	72.0	77.6	69.7

A.6 Robustness of MIXLORA Towards Different Router Loss Coefficient

Table 9: Accuracy comparison of MIXLORA and MIXDORA with different Router Loss for LLaMA2-7B on the commonsense reasoning tasks.

PEFT Method	Router Loss Coef.	ARC-e	ARC-c	BoolQ	OBQA	Avg.
MixLoRA	-	75.5	55.5	72.8	78.8	70.5
	1e-3	77.7	58.1	72.7	84.4	**73.2**
	1e-2	77.0	56.4	73.1	80.6	71.8
	1e-1	76.6	55.7	72.7	80.8	71.5
MixDoRA	-	77.7	56.9	72.8	79.2	71.5
	1e-3	77.5	58.2	72.6	80.9	**72.3**
	1e-2	77.6	56.2	73.0	80.6	71.9
	1e-1	77.3	54.6	72.0	79.8	70.9

A.7 Optimization Algorithm

Algorithm 1 Optimal Forward Propagation of MIXLORA

Require: multi-task token sequence $\mathbf{T}^{l-1}$: $(\mathtt{M}, \mathtt{B}, \mathtt{N}, \mathtt{D})$
Ensure: multi-task token sequence $\mathbf{T}^{l}$: $(\mathtt{M}, \mathtt{B}, \mathtt{N}, \mathtt{D})$
1: /* Allocate multi-task sequences to various MIXLORAs */
2: **for** t in {multi-task sequences $\mathbf{T}^{l-1}$} **do**
3: $\quad\mathbf{T}_t^{l-1}$: $(\mathtt{B}, \mathtt{N}, \mathtt{D}) \leftarrow \mathbf{T}^{l-1}[t, :, :, :]$
4: $\quad\mathbf{r}_t$: $(\mathtt{B} \times \mathtt{N}, \mathtt{K}) \leftarrow \mathbf{Linear}_t(\mathbf{T}_t^{l-1})$
5: $\quad\mathbf{r}'_t$: $(\mathtt{B} \times \mathtt{N}, \mathtt{K}) \leftarrow \mathbf{Norm}(\mathbf{Top2}(\mathbf{Softmax}(\mathbf{r}_t)))$
6: $\quad$/* Reduction of duplicative calculations */
7: $\quad\bar{\mathbf{h}}_t^{W_1}$: $(\mathtt{B}, \mathtt{N}, \mathtt{D}') \leftarrow \mathbf{Linear}_l^{W_1}(\mathbf{T}_t^{l-1})$
8: $\quad\bar{\mathbf{h}}_t^{W_3}$: $(\mathtt{B}, \mathtt{N}, \mathtt{D}') \leftarrow \mathbf{Linear}_l^{W_3}(\mathbf{T}_t^{l-1})$
9: $\quad$/* Calculate the outputs of LoRA experts */
10: $\quad$**for** k in {LoRA experts $\mathbf{E}_t^l$} **do**
11: $\quad\quad\hat{\mathbf{h}}_t^{W_1}$: $(\mathtt{B}, \mathtt{N}, \mathtt{D}') \leftarrow \bar{\mathbf{h}}_t^{W_1} + \mathbf{LoRA}_k^{W_1}(\mathbf{T}_t^{l-1})$
12: $\quad\quad\hat{\mathbf{h}}_t^{W_3}$: $(\mathtt{B}, \mathtt{N}, \mathtt{D}') \leftarrow \bar{\mathbf{h}}_t^{W_3} + \mathbf{LoRA}_k^{W_3}(\mathbf{T}_t^{l-1})$
13: $\quad\quad\hat{\mathbf{h}}_t$: $(\mathtt{B}, \mathtt{N}, \mathtt{D}') \leftarrow \mathbf{SiLU}(\hat{\mathbf{h}}_t^{W_1}) \bigodot \hat{\mathbf{h}}_t^{W_3}$
14: $\quad\quad\mathbf{h}_t$: $(\mathtt{B}, \mathtt{N}, \mathtt{D}) \leftarrow \mathbf{Linear}_l^{W_2}(\hat{\mathbf{h}}_t) + \mathbf{LoRA}_k^{W_2}(\hat{\mathbf{h}}_t)$
15: $\quad\quad\mathbf{T}_t^l$: $(\mathtt{B}, \mathtt{N}, \mathtt{D}) \leftarrow \mathbf{T}_t^l + \mathbf{h}_t \bigodot \mathbf{r}'_t[:, k]$
16: $\quad$**end for**
17: $\quad\mathbf{T}^l$: $(\mathtt{t}+\mathtt{1}, \mathtt{B}, \mathtt{N}, \mathtt{D}) \leftarrow \mathbf{concat}(\mathbf{T}^l, \mathbf{T}_t^l.\mathbf{unsqueeze}(0))$
18: **end for**Return: $\mathbf{T}^l$

By combining two optimization strategies mentioned in Section 3.3, reducing computational complexity (I) and multi-model high-throughout training (II), we propose a forward propagation algorithm described in Algorithm 1. Specifically, the multi-task input sequences $\mathbf{T}^{l-1}$ include various token sequences from M tasks, where each token sequence is sequentially allocated to different MIXLORA modules for processing (line 1). Given that the pretrained dense model weights remain frozen, it becomes feasible to maintain two or more MIXLORA models that share the same pretrained dense model weights. This approach reduces GPU memory cost and improves training efficiency by allowing multiple MIXLORA modules to be trained on a single GPU and reducing kernel launch time. Next, we linearly project the token sequences of task t to the logits $\mathbf{r}_t$ (line 4) and compute the normalized logits $\mathbf{r}'_t$ of activated experts by employing Softmax and Top-2 functions (line 5). We observe that the shared FFN sublayer repeatedly participates in the computation in multiple LoRA experts, which can be avoided with the linear layer W_1 and the linear layer W_3 of the FFN. Therefore, the projected token sequences $\bar{\mathbf{h}}_t^{W_1}$ and $\bar{\mathbf{h}}_t^{W_3}$ are saved in advance before computing the outputs of each expert (lines 7 and 8). Finally, we compute the product of the k-th LoRA metrics $LoRA_k$ plus the shared FFN weights as the weights of the k-th expert and weight the outputs of all activated experts with the logits $\mathbf{r}'_t$ generated by the router to get the output token sequence of the t-th task (line 15).